REGISTRO DE BANCO DE DADOS - SEMANA 25

RELATÓRIO DE ATIVIDADE PRÁTICA

Implementação de Sharding no MongoDB para Escalabilidade

Empresa fictícia: ShopWide

1. Objetivo da atividade

O objetivo desta atividade é compreender e demonstrar o funcionamento do sharding no MongoDB, uma técnica utilizada para distribuir dados entre diferentes servidores, melhorando a escalabilidade e a capacidade de processamento de grandes volumes de informações.

2. Estudo conceitual

2.1. O que é sharding no MongoDB?

Sharding é uma técnica que divide os dados de uma coleção em partes menores, distribuídas entre diferentes servidores chamados shards. Dessa forma, o banco de dados consegue armazenar e processar grandes quantidades de informações sem depender de apenas um servidor.

2.2. Como o sharding melhora a escalabilidade e o desempenho?

O sharding distribui os dados e a carga de trabalho entre vários servidores, permitindo ampliar a capacidade de armazenamento e processamento. Isso ajuda a atender um número maior de consultas e transações, sendo útil para empresas de e-commerce como a ShopWide.

2.3. Qual é a diferença entre sharding e replicação?

O sharding distribui diferentes partes dos dados entre servidores, enquanto a replicação mantém cópias dos mesmos dados em vários servidores. O sharding favorece a escalabilidade horizontal, enquanto a replicação contribui para a disponibilidade e a tolerância a falhas.

3. Configuração do ambiente de sharding

O ambiente de sharding é composto por três elementos principais:

- **mongos:** roteador responsável por receber as consultas e encaminhá-las aos shards.
- **Config servers:** armazenam os metadados necessários para o funcionamento do cluster.
- **Shards:** armazenam as partes dos dados distribuídos.

Para uma configuração real, os servidores precisam ser iniciados corretamente, com os replica sets necessários, e o `mongos` deve apontar para os config servers configurados. O ambiente pode ser executado localmente ou por meio do Docker.

Comando para conectar ao roteador do cluster:

```
mongosh "mongodb://localhost:27017"
```

Habilitação do sharding no banco de dados:

```
sh.enableSharding("shopwide");
```

Definição da chave de shard para a coleção pedidos:

```
sh.shardCollection(
  "shopwide.pedidos",
  { clienteId: "hashed" }
);
```

A chave `clienteId` utiliza o tipo `hashed`, que distribui os documentos conforme o valor hash dessa chave. O comando deve ser executado depois que os shards estiverem adicionados ao cluster.

4. Teste de inserção e monitoramento

Para testar a coleção `pedidos`, pode-se utilizar o seguinte código no MongoDB Shell (`mongosh`):

```
use shopwide

for (let i = 1; i <= 1000; i++) {
  db.pedidos.insertOne({
    clienteId: i,
    produto: "item" + i,
    valor: i * 10
  });
}
```

Resultado esperado: inserção de mil documentos na coleção `pedidos`.

Para verificar a quantidade de documentos inseridos:

```
db.pedidos.countDocuments();
```

Resultado esperado:

```
1000
```

Para consultar o estado do cluster:

sh.status();

Esse comando apresenta informações sobre os shards cadastrados, as coleções fragmentadas e a distribuição dos dados.

A inserção dos documentos, por si só, não garante a distribuição imediata entre os shards. A distribuição depende do funcionamento do cluster, dos chunks e do balanceador.

5. Gerenciamento de shards

5.1. Como adicionar novos shards ao cluster?

É necessário iniciar um novo servidor ou replica set configurado como shard e adicioná-lo pelo `mongos`, utilizando o comando `sh.addShard()` com o endereço correto.

Exemplo:

sh.addShard("shard3/localhost:27023");

Esse exemplo pressupõe a existência de um replica set chamado `shard3`, acessível pelo endereço informado.

5.2. O que acontece se um shard falhar?

Se um shard falhar e não possuir réplicas disponíveis, parte dos dados poderá ficar inacessível. Quando o shard utiliza um replica set com membros saudáveis, outro membro elegível pode assumir as operações, ajudando a manter a disponibilidade.

5.3. Como o MongoDB redistribui os dados ao adicionar shards?

O balanceador do MongoDB pode migrar chunks entre os shards para distribuir os dados de maneira mais equilibrada. Esse processo ocorre conforme as regras do cluster e pode levar algum tempo, não acontecendo necessariamente de forma imediata.

6. Registro dos testes e capturas de tela

As capturas devem ser obtidas durante a execução real dos comandos. As legendas abaixo indicam o que deve aparecer em cada evidência.

Print 1 — Inicialização do ambiente

```
➜ ~ docker compose ps
no configuration file provided: not found
➜ ~
➜ ~ cd ~/shopwide-sharding
➜ shopwide-sharding docker compose ps
NAME                          IMAGE       COMMAND                  SERVICE   CREATED          STATUS
      PORTS
shopwide-sharding-config-1    mongo:8.0   "docker-entrypoint.s…"   config    26 minutes ago   Up 26 min
utes   27017/tcp, 0.0.0.0:27019->27019/tcp, [::]:27019->27019/tcp
shopwide-sharding-mongos-1    mongo:8.0   "docker-entrypoint.s…"   mongos    26 minutes ago   Up 26 min
utes   0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
shopwide-sharding-shard1-1    mongo:8.0   "docker-entrypoint.s…"   shard1    26 minutes ago   Up 26 min
utes   27017/tcp, 0.0.0.0:27018->27018/tcp, [::]:27018->27018/tcp
shopwide-sharding-shard2-1    mongo:8.0   "docker-entrypoint.s…"   shard2    26 minutes ago   Up 26 min
utes   27017/tcp, 0.0.0.0:27020->27018/tcp, [::]:27020->27018/tcp
➜ shopwide-sharding
```

Print 2 — Adição dos shards

```
1
➜ shopwide-sharding cd ~/shopwide-sharding
docker compose exec -T config mongosh --port 27019 --eval 'rs.status().myState'
1
➜ shopwide-sharding docker compose exec -T shard1 mongosh --port 27018 --eval 'rs.status().myState'
1
➜ shopwide-sharding docker compose exec -T shard2 mongosh --port 27018 --eval 'rs.status().myState'
1
```

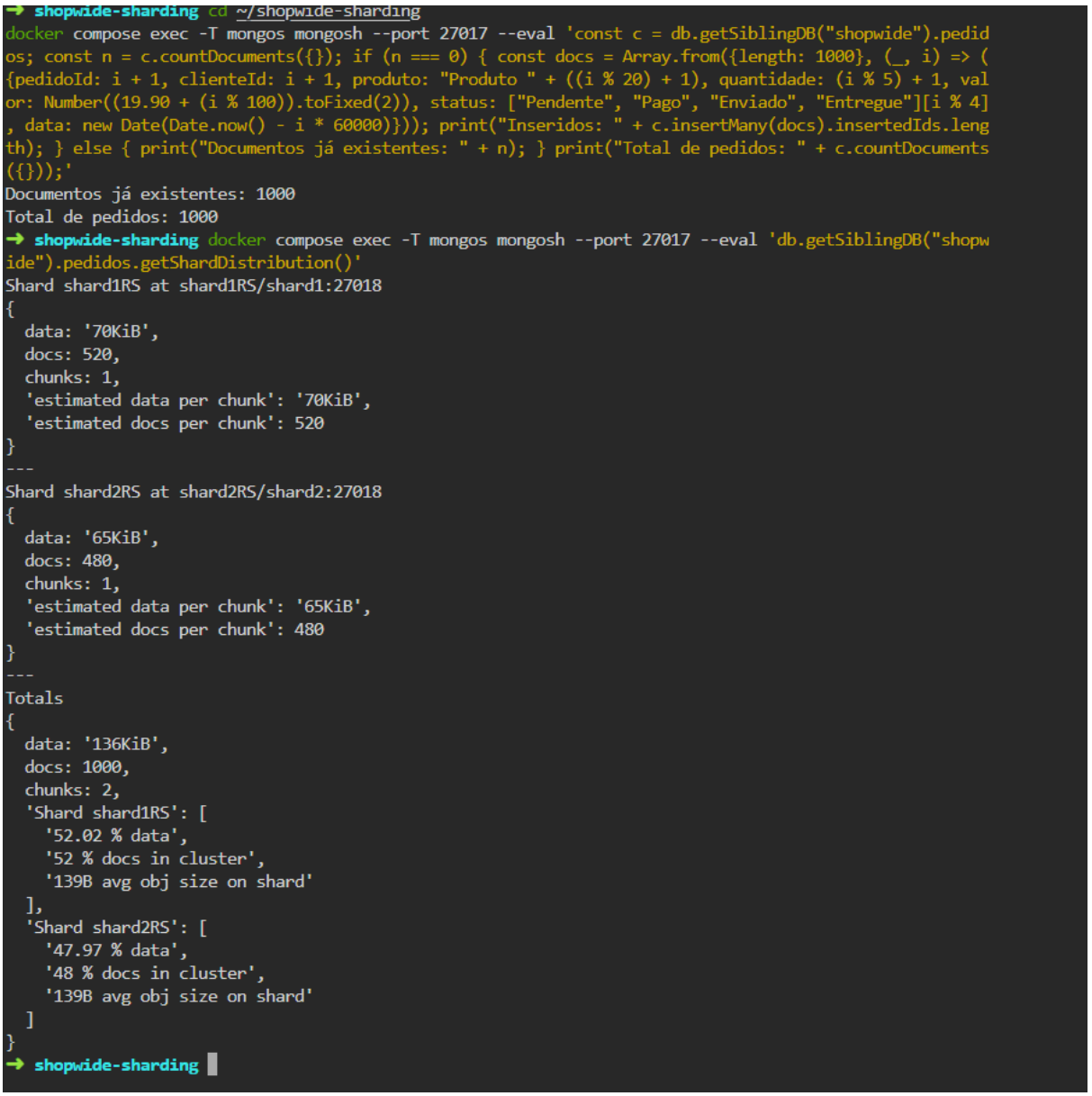
Print 3 — Configuração do sharding

```
→ shopwide-sharding cd ~/shopwide-sharding
docker compose exec -T mongos mongosh --port 27017 --eval 'sh.status()'
shardingVersion
{ _id: 1, clusterId: ObjectId('6adf91060de03b8673af4a6a') }
---
shards
[
  {
    _id: 'shard1RS',
    host: 'shard1RS/shard1:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1793036754, i: 10 }),
    replSetConfigVersion: Long('-1')
  },
  {
    _id: 'shard2RS',
    host: 'shard2RS/shard2:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1793036764, i: 9 }),
    replSetConfigVersion: Long('-1')
  }
]
---
active mongoses
[ { '8.0.34': 1 } ]
---
autosplit
{ 'Currently enabled': 'yes' }
---
balancer
{
  'Currently enabled': 'yes',
  'Failed balancer rounds in last 5 attempts': 0,
  'Currently running': 'no',
  'Migration Results for the last 24 hours': 'No recent migrations'
}
---
shardedDataDistribution
[
  {
    ns: 'config.system.sessions',
    shards: [
      {
        shardName: 'shard1RS',
        numOrphanedDocs: 0,
        numOwnedDocuments: 64,
        ownedSizeBytes: 6336,
        orphanedSizeBytes: 0
      }
    ]
  },
  {
    ns: 'shopwide.pedidos',
```

Print 4 — Inserção dos documentos

```
  }
]
---
databases
[
  {
    database: { _id: 'config', primary: 'config', partitioned: true },
    collections: {
      'config.system.sessions': {
        shardKey: { _id: 1 },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [ { shard: 'shard1RS', nChunks: 1 } ],
        chunks: [
          { min: { _id: MinKey() }, max: { _id: MaxKey() }, 'on shard': 'shard1RS', 'last modified': T
imestamp({ t: 1, i: 0 }) }
        ],
        tags: []
      }
    }
  },
  {
    database: {
      _id: 'shopwide',
      primary: 'shard1RS',
      version: {
        uuid: UUID('451ad9d3-feb3-4378-9403-87a55598fdce'),
        timestamp: Timestamp({ t: 1793036876, i: 2 }),
        lastMod: 1
      }
    },
    collections: {
      'shopwide.pedidos': {
        shardKey: { clienteId: 'hashed' },
        unique: false,
        balancing: true,
        allowMigrations: true,
        chunkMetadata: [
          { shard: 'shard1RS', nChunks: 1 },
          { shard: 'shard2RS', nChunks: 1 }
        ],
        chunks: [
          { min: { clienteId: MinKey() }, max: { clienteId: Long('0') }, 'on shard': 'shard2RS', 'last
 modified': Timestamp({ t: 1, i: 0 }) },
          { min: { clienteId: Long('0') }, max: { clienteId: MaxKey() }, 'on shard': 'shard1RS', 'last
 modified': Timestamp({ t: 1, i: 1 }) }
        ],
        tags: []
      }
    }
  }
]
```

Print 5 — Monitoramento do cluster



7. Desafios e observações

A configuração de um cluster de sharding exige atenção às portas, aos caminhos dos dados, à configuração dos replica sets e à comunicação entre os componentes. Também é necessário garantir que os comandos sejam executados por meio do `mongos` e que a chave de shard seja adequada aos dados da aplicação.

Um desafio adicional é verificar se os dados estão realmente distribuídos entre os shards, pois a simples inserção de documentos não comprova, sozinha, que o balanceamento ocorreu.

8. Conclusão

A implementação de sharding no MongoDB é uma solução importante para empresas que precisam lidar com o crescimento contínuo de dados e transações. No caso da ShopWide, a

distribuição dos pedidos entre diferentes servidores pode ampliar a capacidade de armazenamento e processamento e reduzir a dependência de uma única máquina.

Conclui-se que o sharding favorece a escalabilidade horizontal, mas exige configuração adequada, escolha criteriosa da chave de shard e monitoramento constante para garantir o funcionamento esperado do cluster.